ID	Defecto	Severidad	Prioridad	Estado
DEF-001	Caché no se invalidaba al crear contacto	Alta	P1	Cerrado
DEF-002	PHP sin extensión PostgreSQL	Alta	P1	Cerrado
DEF-003	Laravel no encontraba tabla contactos inicialmente	Alta	P1	Cerrado
DEF-004	Método de caché marcado incorrectamente por IDE	Baja	P3	Cerrado
DEF-005	Consulta individual inexistente no devolvía Problem Details	Media	P2	Cerrado

DEF-001 — Controlador de PostgreSQL no disponible

ID: DEF-001
Título: PHP no podía establecer conexión con PostgreSQL.
Módulo afectado: Persistencia / Configuración del backend.
Tipo: Configuración / Entorno.
Severidad: Alta.
Prioridad: P1.
Estado: Cerrado.

Descripción:
Durante la configuración de PostgreSQL, Laravel no podía utilizar correctamente la conexión pgsql debido a que las extensiones necesarias de PHP no se encontraban habilitadas.

Pasos para reproducir:

1. Configurar DB_CONNECTION=pgsql en .env.
2. Ejecutar una operación de Laravel que requiera acceso a la base de datos.
3. La aplicación no logra establecer correctamente la conexión con PostgreSQL.

Resultado esperado:
Laravel debe conectarse a PostgreSQL utilizando el controlador pgsql.

Resultado obtenido:
La conexión no podía realizarse debido a la configuración de las extensiones de PHP.

Causa identificada:
Las extensiones pdo_pgsql y pgsql no estaban habilitadas en PHP.

Solución aplicada:
Se habilitaron las extensiones correspondientes en php.ini y se reinició el entorno de ejecución.

Resultado de verificación:
Laravel logró utilizar PostgreSQL correctamente y posteriormente ejecutar las migraciones.

DEF-002 — Error de autenticación con PostgreSQL

ID: DEF-002
Título: Fallo de autenticación del usuario PostgreSQL.
Módulo afectado: Base de datos / Configuración.
Tipo: Configuración.
Severidad: Alta.
Prioridad: P1.
Estado: Cerrado.

Descripción:
Durante la configuración de la nueva base de datos, no era posible autenticar correctamente al usuario postgres, lo que impedía que Laravel accediera al motor de persistencia.

Pasos para reproducir:

1. Configurar PostgreSQL como conexión principal.
2. Introducir las credenciales disponibles.
3. Intentar establecer una conexión con la base de datos.

Resultado esperado:
El usuario debe autenticarse y permitir el acceso a la base de datos modulocrm_pg.

Resultado obtenido:
PostgreSQL rechazaba la autenticación.

Causa identificada:
La contraseña disponible para el usuario de PostgreSQL no podía ser utilizada para establecer correctamente la conexión.

Solución aplicada:
Se restableció la contraseña del usuario administrador de PostgreSQL y posteriormente se configuró la nueva credencial en .env.

Resultado de verificación:
Laravel logró establecer conexión correctamente con modulocrm_pg.

DEF-003 — Tablas del CRM inexistentes en PostgreSQL

ID: DEF-003
Título: Las tablas requeridas por el CRM no estaban disponibles en PostgreSQL.
Módulo afectado: Persistencia.
Tipo: Base de datos / Despliegue.
Severidad: Alta.
Prioridad: P1.
Estado: Cerrado.

Descripción:
Después de cambiar la persistencia del proyecto a PostgreSQL, inicialmente la nueva base de datos no contenía las estructuras necesarias para operar el Módulo CRM.

Pasos para reproducir:

1. Crear la base de datos modulocrm_pg.
2. Configurar Laravel para utilizarla.
3. Intentar consultar información antes de ejecutar las migraciones.

Resultado esperado:
Las tablas necesarias deben existir para poder ejecutar las operaciones del CRM.

Resultado obtenido:
La base de datos no disponía inicialmente de las tablas del sistema.

Causa identificada:
Las migraciones todavía no se habían ejecutado en la nueva instancia de PostgreSQL.

Solución aplicada:
Se verificaron primero mediante:

php artisan migrate --pretend

y posteriormente se ejecutaron:

php artisan migrate

Resultado de verificación:
Las tablas users, contactos, clientes, proveedores, empleados, entre otras, fueron creadas correctamente y aparecieron como Ran en migrate:status.

DEF-004 — Configuración de caché incompleta

ID: DEF-004
Título: Operaciones de caché no disponibles antes de completar la configuración.
Módulo afectado: Caché / Redis.
Tipo: Configuración / Dependencia.
Severidad: Media.
Prioridad: P2.
Estado: Cerrado.

Descripción:
Durante la integración de la estrategia Cache-Aside se detectó que la aplicación todavía no disponía de toda la configuración necesaria para utilizar Redis como almacenamiento de caché.

Pasos para reproducir:

1. Configurar Laravel para utilizar Redis.

2. Intentar ejecutar operaciones de caché sin haber instalado/configurado el cliente correspondiente.
3. La aplicación no puede utilizar Redis correctamente.

Resultado esperado:
Laravel debe poder almacenar y recuperar información desde Redis.

Resultado obtenido:
El entorno PHP no disponía inicialmente del cliente necesario para utilizar Redis desde Laravel.

Causa identificada:
La extensión nativa de Redis para PHP no se encontraba disponible en el entorno utilizado.

Solución aplicada:
Se instaló Predis mediante Composer:

composer require predis/predis

y se configuró:

REDIS_CLIENT=predis

CACHE_STORE=redis

Resultado de verificación:
La operación de escritura y lectura mediante Cache::store('redis') funcionó correctamente y Redis respondió PONG mediante redis-cli.

DEF-005 — Riesgo de información obsoleta después de modificar contactos

ID: DEF-005
Título: Necesidad de invalidar la caché después de operaciones de escritura.
Módulo afectado: Contactos / Redis.
Tipo: Consistencia / Caché.
Severidad: Alta.
Prioridad: P1.
Estado: Cerrado.

Descripción:
Durante la implementación de Cache-Aside se identificó que una operación de creación o modificación de contactos debe eliminar la información almacenada previamente en Redis para evitar entregar listados obsoletos.

Condición de prueba:

1. Ejecutar GET /api/v1/contactos.
2. Obtener y almacenar el listado en Redis.
3. Ejecutar nuevamente el GET y obtener X-Cache: HIT.

4. Crear un contacto mediante POST /api/v1/contactos.
5. Consultar nuevamente el listado.

Resultado esperado:
Después del POST, la caché debe invalidarse y la siguiente consulta debe producir X-Cache: MISS con la información actualizada.

Riesgo identificado:
Sin invalidación, Redis podría continuar devolviendo la versión anterior del listado.

Causa:
La estrategia Cache-Aside necesita invalidación explícita después de las operaciones que modifican información.

Solución aplicada:
Después del registro de un contacto se implementó:

$this->cacheManager->store('redis')->clear();

Resultado de verificación:

GET → MISS

GET → HIT

POST → 201 Created

GET → MISS

GET → HIT

El comportamiento esperado fue comprobado correctamente.